24H365D自律開発運用 取扱説明書

～ローカル環境でのゼロからの環境構築・起動・日常運用・障害保守完全マニュアル～

オープンソース推進部
システム統制課
N-Y-Public

図書番号：D01-000002

N-Y-Public

目次

第1章 環境導入と初期セットアップ 3
第2章 デーモン起動と自律開発運用 9
第3章 障害保守と運用チェックリスト 16
第4章 起動引数とパラメータ解説 20

- 第1章 環境導入と初期セットアップ
  - 24H365D自律開発運用の全体像とアーキテクチャ
  - 【導入①】ローカル前提ツールの準備とクローン
  - 【導入②】環境変数（.env）とGitHub認証の設定
  - 【導入③】Git匿名化設定とPre-Commitフックの初期化
  - 【導入④】Dockerコンテナ環境の起動とヘルスチェック
- 第2章 デーモン起動と自律開発運用
- 第3章 障害保守と運用チェックリスト
- 第4章 起動引数とパラメータ解説

© 2026 N-Y-Public

24H365D自律開発運用の全体像とアーキテクチャ

- ホスト・Docker・Antigravity・GitHubの4層が連携して自律稼働
  - 完全な環境隔離と相互牽制により、無人環境下でも安全性を死守

自律開発基盤を構成する4つの責務レイヤー

ホストOS & Dockerサンドボックス

- ホストOS（開発者）: 非同期指示の入力、日報・週報の確認、全体監視
- Dockerサンドボックス: `marp-slide-mcp-app` による隔離環境
- コマンド実行: すべて `dev.ps1` または `docker compose run` 経由
- 汚染防止: ホスト環境にNode/Puppeteer依存を一切残さない

常駐Pythonオーケストレーター & ステートレスAIワーカー統制

- 常駐オーケストレーター: `autonomous_daemon.py` が主導制御と時計監視を担当
- ステートレスAIワーカー: 1工程ごとに新規 `agy` セッションを起動・破棄
- マルチエージェント: 起票（`ROLE-PM`）と独立審査（`ROLE-ARCH`）を完全分離
- 品質ゲート: `verify_rtm.py` による4層トレーサビリティ完全保証

出所: `docs/00_project_charter/01_project_charter.md` および `scripts/autonomous_daemon.py`

© 2026 N-Y-Public

【導入①】ローカル前提ツールの準備とクローン

- DockerやGit等の前提ツールを揃え、安全にリポジトリをクローン
  - ホスト環境を汚染せず、Dockerコンテナ内で開発環境を完結

STEP 1（前提要件の確認・リポジトリ取得・作業ディレクトリ初期化）

前提ツールと動作要件

- Docker Desktop: WSL2 / Colima / Docker Engine (稼働必須)
- Git 2.30+: バージョン管理・Worktree操作
- Antigravity 2.0: 自律エージェント稼働環境
- Python 3.10+: ホスト側補助スクリプト（基本Docker内完結）
- OS対応: Windows 11 / macOS / Ubuntu Linux

クローンと作業ディレクトリ移動

```
# 1. リポジトリをローカルへクローン
git clone https://github.com/N-Y-Public/marp-slide-mcp.git

# 2. リポジトリルートへ移動
cd marp-slide-mcp

# 3. リモート最新ブランチの確認
git status
```

- ※ホスト側での `npm install` は不要（Docker内で完結）

出所: `AGENTS.md`（第1節 Docker Development Environment Only）

© 2026 N-Y-Public

【導入②】環境変数（.env）とGitHub認証の設定

- .envにトークンを設定し、事前アカウント検証ゲートを有効化
  - 期待アカウントと異なる場合の外部送信を物理的に完全遮断

STEP 2（.envテンプレート複製・PATトークン発行・Identity防壁）

環境変数の役割と防壁仕様

- `.env` 作成: `.env.example` からローカル複製
- 事前アカウント検証ゲート: GitHub API呼び出し直前に自動照合
- アカウントID照合: `330902043`（`N-Y-Public` 専用ID）
- 物理的遮断: 不一致時は直ちに `IDENTITY-GATE-ABORT`
- 除外設定: `.env` は `.gitignore` でリポジトリ外管理

`.env` 設定手順とコマンド

```
# 1. テンプレートから設定ファイルを複製
cp .env.example .env

# 2. .env 内に以下を定義（エディタで編集）
GITHUB_PAT_MARP_SLIDE_MCP=ghp_YourGitHubPersonalAccessToken
EXPECTED_GITHUB_LOGIN=N-Y-Public
EXPECTED_GITHUB_ID=330902043
```

- PATスコープ: `repo`, `read:org` を付与

出所: `docker-compose.yml` および `scripts/autonomous_dev_loop.py`

© 2026 N-Y-Public

【導入③】Git匿名化設定とPre-Commitフックの初期化

- Gitコミッターを匿名化し、コミット時の機密・絶対パス混入を阻止
  - Q-09統制により個人実名やホスト環境依存コードをゼロに維持

STEP 3（ローカルGit Identity設定・Pre-Commitフック登録）

OSSプライバシー保護基準 (Q-09)

- コミッター匿名化: 実名・個人メールの混入を絶対禁止
- Git Identity固定: `N-Y-Public` ノーリプライアドレス
- フックによる自動検査: コミット実行時に機械的に遮断
- 検査対象: 個人実名、`/Users/...` や `C:¥...` 等の絶対パス
- 違反時挙動: エラーを出力しコミットを即時アボート

初期化コマンドリファレンス

```
# 1. ローカルリポジトリ限定でコミッター情報を固定
git config user.name "N-Y-Public"
git config user.email "330902043+N-Y-Public@users.noreply.github.com"

# 2. Pre-Commitフックの登録
git config core.hooksPath .githooks

# 3. Linux/Mac環境での実行権限付与
chmod +x .githooks/pre-commit
```

出所: `AGENTS.md`（第2節 OSS Privacy & Identity Protection）および `.githooks/pre-commit`

© 2026 N-Y-Public

【導入④】Dockerコンテナ環境の起動とヘルスチェック

- dev.ps1またはdev.shでコンテナを起動し、初期健全性を確認
  - TypeScriptビルドおよび自動テストのパスで稼働準備を完了

STEP 4（DevContainer起動・疎通確認・初期品質ゲート通過）

コンテナ起動プロトコル

- ヘルパースクリプト: OS別の起動コマンドを用意
- イメージ自動構築: Node 22 / Chromium / 日本語フォント
- バックグラウンド実行: `-d` オプションで常駐稼働
- ヘルスチェック: ビルド・単体テストを即時実行
- 準備完了判定: 全テスト合格（Exit 0）でセットアップ完了

起動とヘルスチェックコマンド

```
# 1. コンテナの起動 (Windows: PowerShell)
.¥dev.ps1 start
#  (Linux / Mac: ./dev.sh start または docker compose up -d)

# 2. 稼働状態の確認
.¥dev.ps1 status

# 3. 初期ヘルスチェック (TypeScriptビルド & テスト)
docker compose run --rm app npm run build
docker compose run --rm app python3 -m unittest discover -s
tests -p "test_*.py"
```

出所: `dev.ps1`, `dev.sh`, `docker-compose.yml`

© 2026 N-Y-Public

- 第1章 環境導入と初期セットアップ
- **第2章 デーモン起動と自律開発運用**
  - 【起動①】常駐Pythonオーケストレーターデーモンの起動
  - 【起動②】時計監視による定時レポートの決定論的自動生成
  - 【運用サイクル】マルチエージェント相互牽制の実行メカニズム
  - 【日常運用①】1日3回の日報確認と指示タグ記入
  - 【日常運用②】毎週水曜16:00の週報による上司層への報告
  - 【日常運用③】Pull Requestの確認とマージ手順
- 第3章 障害保守と運用チェックリスト
- 第4章 起動引数とパラメータ解説

© 2026 N-Y-Public
CONFIDENTIAL

【起動①】常駐Pythonオーケストレーターデーモンの起動

- 常駐オーケストレーター（autonomous_daemon.py）が主導して連続消化
  - 1工程ごとに新規AIセッションを起動・破棄しコンテキスト枯渇を防止

STEP 5（常駐監視デーモン起動・ステートレスAIワーカー呼び出し）

常駐デーモン主導の制御プロトコル

- GitHub自動監視: `--issues-json` 省略時はGitHubのOpen Issueを自動取得
- 並行度1件統制: デフォルトで1度に並行着手するIssueを1件（`1`）に制限
- ステートレス実行: Stage A・審査・Stage Bごとに新規セッション起動・破棄
- DoD自動検証: ビルド・テスト・`verify_rtm.py` 合格時のみPR生成
- 次Issue自動遷移: 1件完了または保留後、停止せず次Issueへ即時遷移

常駐オーケストレーター起動コマンド

```
# 1. 常駐デーモンの起動（GitHub自動監視・並行上限1件・30秒周期）
docker compose run --rm app python3 scripts/autonomous_daemon.py
--poll-interval 30

# 2. 1サイクル動作確認（--once / 並行数はデフォルト1件）
docker compose run --rm app python3 scripts/autonomous_daemon.py
--once --max-concurrent-issues 1

# ※PowerShellでの実行例（改行時は `¥` ではなくバッククォートを
使用）：
# .¥dev.ps1 run python3 scripts/autonomous_daemon.py --poll-
interval 30
```

出所: `scripts/autonomous_daemon.py` および `scripts/autonomous_dev_loop.py`

© 2026 N-Y-Public

【起動②】時計監視による定時レポートの決定論的自動生成

- AIを介さずPython時計監視により日報3回と水曜週報を定時出力
  - トークン消費ゼロかつ遅延ゼロでdocs/reports/配下へ確実に出力

STEP 6（JST時計監視・二重出力防止・手動レポート生成コマンド）

定時レポートの決定論的配信仕様

- 朝刊（`07:30 JST`）: 夜間の自律走行成果と保留判断事項
- 昼刊（`11:50 JST`）: 午前の進捗状況と午後の予定
- 夕刊（`18:00 JST`）: 日中帯の完了タスクと夜間走行キュー
- 週報（`水曜 16:00 JST`）: 上司層向け「課題・解決・結果・宿題」
- 二重出力防止: `emitted_reports` 記録により同一枠の重複生成を抑止

定時自動監視と手動即時出力コマンド

```
# 常駐デーモン起動中は 07:30 / 11:50 / 18:00 / 水曜16:00 に自動出力
# 手動で特定スロットのレポートを即時生成する場合:

# 朝刊・昼刊・夕刊日報の手動出力
docker compose run --rm app python3 scripts/generate_reports.py --time 07:30 --mode daily
docker compose run --rm app python3 scripts/generate_reports.py --time 11:50 --mode daily
docker compose run --rm app python3 scripts/generate_reports.py --time 18:00 --mode daily

# 定例週報＋日報の手動出力（水曜 16:00 枠）
docker compose run --rm app python3 scripts/generate_reports.py --time 16:00 --mode both
```

出所: `scripts/autonomous_config.json`, `scripts/autonomous_daemon.py`, `scripts/generate_reports.py`

© 2026 N-Y-Public

【運用サイクル】マルチエージェント相互牽制の実行メカニズム

- 起票者と審査者を分離し、自己承認や曖昧要件の実装着手を阻止
  - 独立エージェントが5大判定基準で厳格にStage A→Bを審査

STEP 7（起票と検証の分離・自己承認拒絶・判定フロー）

相互牽制プロトコル (Q-01 / Q-11)

- 起票ロール（`ROLE-PM`）: PBIを作成し受入基準を定義
- 審査ロール（`ROLE-ARCH`）: 独立した視点で品質を客観審査
- 自己承認拒絶: 起票者と審査者が同一IDの場合は自動拒否
- 5大判定基準: 構文規約・GWT検証性・網羅性・仮定・Q-09
- 判定ステータス: `PASS`（着手）/ `REJECT`（差戻）/ `ESCALATE`（日報へ）

ゲート審査の手動実行コマンド

```
# 独立審査エージェントによるゲート審査（1行実行）
docker compose run --rm app python3
scripts/autonomous_dev_loop.py review-gate --pbi
docs/backlog/todo/PBI-019_xxx.md --drafter pbi-drafter-agent --
reviewer stage-gate-reviewer-agent
```

- 合格時: `docs/backlog/in_progress/` へ移動し実装へ進む

出所: `.agents/rules/ipa-process-gate.md` および `scripts/autonomous_dev_loop.py`

© 2026 N-Y-Public

【日常運用①】1日3回の日報確認と指示タグ記入

- メンテナーは朝昼夕の数分で日報を確認し、指示タグでAIを誘導
  - 非同期指示（APPROVE/REJECT）だけで手戻りを未然に防止

STEP 8（日報チェック・コメント記入・自律ループへの反映）

日報確認ルーティン (数分で完了)

- 確認先: `docs/reports/daily/YYYY-MM-DD_{slot}.md`
- 確認項目: 直近完了PBI、進行中タスク、判断依頼事項
- 保留事項: `ASM-NNN` 等の未確定仕様がないか確認
- 非同期舵取り: コメント欄にタグを追記して保存
- 手戻りゼロ: AIの暴走や勝手な仕様捏造を事前に遮断

指示タグの記入例と反映コマンド

```
## 5. コメント記入欄（メンテナー記入用）
- **指示:** [APPROVE: PBI-019, ASM-001]
- **指示:** [REJECT: PBI-020]
- **指示:** [PRIORITY: #25, #22]
```

```
# 指示コメントを自律ループの状態へ即時反映（1行実行）
docker compose run --rm app python3
scripts/autonomous_dev_loop.py apply-feedback --report
docs/reports/daily/2026-11-08_morning.md
```

出所: `docs/templates/tpl_96_daily_report.md` および `scripts/autonomous_dev_loop.py`

© 2026 N-Y-Public

【日常運用②】毎週水曜16:00の週報による上司層への報告

- 毎週水曜16時に「課題・解決・結果・宿題」を上司層へ1枚報告
  - コード詳細は省き、意思決定と進捗把握に特化したハイレベル報告

STEP 9（週報テンプレート構造とプロジェクト統括への定例報告）

構成セクション	対象読者が知りたい問い	報告内容・記述粒度
1. エグゼクティブサマリー	プロジェクト健全性は順調か？	全体ステータス（Green/Yellow/Red）と今週の要約
2. 課題・宿題と解決結果	何が課題で、どう解決し結果どうなったか？	**「①課題/先週宿題」「②解決策」「③結果」**の一覧表
3. 新たな宿題事項と期限	次にいつまでに何を完了させるのか？	**「新たな宿題」「発生背景」「担当」「完了期限」**の表
4. 健全性指標推移	計画通りに課題が消化されているか？	完了PBI累計（増減）、残課題数、品質適合率（100%）
5. マネジメント相談事項	上司の意思決定が必要な事項はあるか？	リソース調整や組織方針に関する相談事項

出所: `docs/templates/tpl_97_weekly_report.md` および `scripts/generate_reports.py`

© 2026 N-Y-Public

【日常運用③】Pull Requestの確認とマージ手順

- 全AC合格かつRTM断絶ゼロのタスクのみ規約準拠PRを発行
  - メンテナーによる最終マージ後、直ちに次タスクへ自律復帰

STEP 10（DoD監査・PR自動起票・マージ・次ループ遷移）

PR起票とマージプロトコル

- 完了定義 (DoD): ビルド成功・全テスト合格・RTM断絶0件
- PRテンプレート: `.github/pull_request_template.md`
- 記載項目: 変更概要・関連PBI・テスト結果・RTM監査証明
- メンテナー確認: GitHub上で差分を確認しマージ承認
- 自動循環: マージ後、AIは即座に次の未対応Issueへ着手

PR発行とマージコマンド

```
# 1. PR発行（テンプレート準拠）
gh pr create --template .github/pull_request_template.md

# 2. CIおよびテスト結果の確認
gh pr checks

# 3. マージ実行（Squash and merge 推奨）
gh pr merge --squash --delete-branch
```

- マージ後、ローカルブランチを最新化してループ継続

出所: `.github/pull_request_template.md` および本リポジトリ開発憲法（AGENTS.md）

© 2026 N-Y-Public

- 第1章 環境導入と初期セットアップ
- 第2章 デーモン起動と自律開発運用
- 第3章 障害保守と運用チェックリスト
  - 【障害対応①】Dockerコンテナ異常・Worktreeロックの復旧
  - 【障害対応②】認証エラーとゲート差し戻しのリカバリ手順
  - 【まとめ】ローカル24H365D自律運用チェックリスト
- 第4章 起動引数とパラメータ解説

© 2026 N-Y-Public

【障害対応①】Dockerコンテナ異常・Worktreeロックの復旧

- コンテナのハングアップやWorktreeロックは専用コマンドで復旧
  - dev.ps1のrestartやunlock機能で短時間での再開を実現

TROUBLE 1（コンテナ停止・応答不全・Worktree競合の解消）

トラブル原因と復旧方針

- コンテナハング: メモリ不足や異常プロセス常駐時に発生
- Worktreeロック: 異常終了時に `.git/worktrees` が残存
- 修復方針: コンテナの再起動および残存ロックの強制クリーン
- 恒久対策: 再起動後も直らない場合はキャッシュなし再ビルド
- 所要時間: ヘルパースクリプトにより1〜2分で復旧完了

復旧コマンドリファレンス

```
# 1. コンテナの再起動（Windows: PowerShell）
.¥dev.ps1 restart

# 2. Git Worktreeロックの強制解除
.¥dev.ps1 unlock-worktrees

# 3. 完全初期化（キャッシュを破棄して再構築）
docker compose down
docker compose build --no-cache
.¥dev.ps1 start
```

出所: `dev.ps1`, `dev.sh` および Docker運用知見

© 2026 N-Y-Public

【障害対応②】認証エラーとゲート差し戻しのリカバリ手順

- アカウント不一致や審査REJECT時は、原因を特定して迅速に対処
  - 日報の指摘事項をもとにPBI受入基準を補正して再審査を通過

TROUBLE 2（IDENTITY-GATEエラー・Stage A審査落ちの復旧）

認証エラー（`IDENTITY-GATE-ABORT`）

- 原因: PATトークンの失効、または別アカウントでログイン中
- 対処1: `.env` の `GITHUB_PAT_MARP_SLIDE_MCP` を更新
- 対処2: `gh auth status` で `N-Y-Public` との一致を確認

ゲート審査落ち（`REJECT`）

- 原因: 禁止曖昧語（「適宜」等）の残存、GWT形式の不備
- 対処: 日報の指摘を読み、PBIのACを定量表現へ修正

再審査と整合性リカバリコマンド

```
# 1. 修正後のPBIを再審査（手動トリガー・1行実行）
docker compose run --rm app python3
scripts/autonomous_dev_loop.py review-gate --pbi
docs/backlog/todo/PBI-xxx.md --drafter pbi-drafter-agent --
reviewer stage-gate-reviewer-agent

# 2. RTM不整合発生時の完全監査実行（1行実行）
docker compose run --rm app python3 .agents/skills/ipa-
traceability-validator/scripts/verify_rtm.py
```

出所: `scripts/autonomous_dev_loop.py` および `.agents/rules/ipa-process-gate.md`

© 2026 N-Y-Public

【まとめ】ローカル24H365D自律運用チェックリスト

- 導入時と日常運用のチェックリストで、安定した自律稼働を死守
  - 人間は非同期の舵取りに徹し、AIが規約準拠の開発を24H継続

導入完了基準と毎日のルーティンチェック

初回セットアップ完了チェック（5項目）

- [ ] Docker環境: `.¥dev.ps1 start` でコンテナ起動完了
- [ ] 環境変数: `.env` に有効なGitHub PATを設定完了
- [ ] Git設定: `user.name: N-Y-Public` に固定完了
- [ ] フック設定: `core.hooksPath .githooks` を有効化
- [ ] 初期健全性: `npm run build` & `verify_rtm.py` がPASS

日常運用ルーティン（3項目）

- [ ] 日報チェック: 朝昼夕の3回、保留事項に `[APPROVE]` 記入
- [ ] 週報チェック: 毎週水曜16時、上司報告の内容を確認
- [ ] PRマージ: 自動起票されたPRをレビューしマージ
- 運用の心得:
  - 人間はコードを書かず、日報での非同期指示に専念
  - AIの自己承認を許さず、品質ゲートを100%維持

出所: 本リポジトリ運用マニュアル（`24H365D自律開発運用取扱説明書`）まとめ

© 2026 N-Y-Public

- 第1章 環境導入と初期セットアップ
- 第2章 デーモン起動と自律開発運用
- 第3章 障害保守と運用チェックリスト
- **第4章 起動引数とパラメータ解説**
  - 【起動引数①】Docker起動構文と主要引数一覧
  - 【起動引数②】常駐デーモン高度オプション一覧
  - 【起動引数③】運用シーン別推奨コマンド集

© 2026 N-Y-Public

【起動引数①】Docker起動構文と主要引数一覧

- Docker Compose構文と常駐デーモン主要引数の役割を完全把握
  - コンテナ隔離・自動破棄・ポーリング周期・並行数上限を体系化

Docker Compose実行引数およびデーモン基本パラメーター一覧

対象区分	引数・構文要素	既定値	役割と実運用時の設定基準
Docker構文	`docker compose run`	-	独立した新規コンテナを生成してコマンドをワンショット実行
Docker引数	`--rm`	-	終了時にコンテナを自動破棄しホストストレージ枯渇を防止
サービス名	`app`	-	`docker-compose.yml` で定義された開発実行コンテナ名
ランタイム	`python3`	-	コンテナ内に隔離されたNode 22/Python 3ハイブリッド環境
デーモン引数	`--poll-interval <秒>`	`0.0`	サイクル間の待機秒数。本番常駐時は `30` を指定しAPI枯渇回避
デーモン引数	`--max-concurrent-issues`	`1`	1度に並行着手する最大Issue数。`1` 件固定で作業混交を防止
デーモン引数	`--issues-json <パス>`	なし	ローカルIssue配列JSON。省略時はGitHub Open Issueを自動取得
デーモン引数	`--once`	`False`	1サイクルのみ実行して即時終了。セットアップ直後の疎通確認用

出所: `scripts/autonomous_daemon.py` および `docker-compose.yml`

© 2026 N-Y-Public

【起動引数②】常駐デーモン高度オプション一覧

- リポジトリ指定やロール分離等の高度引数を使い分け、柔軟な運用を実現
  - 本番常駐・初期疎通・オフライン検証の目的に応じた最適設定を網羅

高度制御引数一覧（監視設定・ロール指定・永続化パス）

引数名 (CLIフラグ)	既定値	設定意味と自律デーモン稼働時の挙動
`--github-repo <リポジトリ名>`	設定ファイル値	監視対象リポジトリ（例: `N-Y-Public/marp-slide-mcp` ）
`--no-github-fetch`	`False`	GitHub API自動取得を無効化。オフライン検証環境で使用
`--max-iterations <回数>`	なし (無制限)	最大実行サイクル数。指定回数を消化後に安全にループ停止
`--state-json <パス>`	`docs/reports/...`	デーモン状態（完了・保留Issue、日報履歴）の永続化先
`--drafter <RoleID>`	`ROLE-PM`	Stage A（PBI起票）を担当するAIエージェントロールID
`--reviewer <RoleID>`	`ROLE-ARCH`	Stage A→B独立審査を担当するエージェントロールID（自己承認防止）
`--quiet`	`False`	コンソールへのリアルタイム進捗ログ表示を抑制（JSON結果のみ出力）
`--repo-root <パス>`	カレントパス	リポジトリルートパスの明示指定

出所: `scripts/autonomous_daemon.py` および `scripts/autonomous_config.json`

© 2026 N-Y-Public

【起動引数③】運用シーン別推奨コマンド集

- 本番24H常駐から疎通テスト・オフライン検証まで目的に応じて使い分け
  - 1行実行およびPowerShell環境での記法を遵守し誤入力を防止

実運用における3大実行パターンとPowerShell利用時の留意点

実運用における主要起動コマンド

- 1. 本番24H365D常駐稼働（標準推奨）:

```
docker compose run --rm app python3
scripts/autonomous_daemon.py --poll-interval 30
```

  - GitHub Open Issueを自動監視し、並行度1件で連続消化

- 2. 初期疎通確認（1サイクル実行）:

```
docker compose run --rm app python3
scripts/autonomous_daemon.py --once
```

  - 時計監視・Issue取得・1工程実行の健全性を即時確認

オフライン検証とシェル記法

- 3. オフライン・ローカルキュー検証:

```
docker compose run --rm app python3
scripts/autonomous_daemon.py --issues-json
docs/reports/issues_queue.json --no-github-fetch
```

- PowerShell実行時の留意点:
  - コマンドは改行せず1行で実行することを推奨
  - 改行して複数行で記述する場合は `¥` ではなくバッククォート（`` ` ``）を使用

出所: `scripts/autonomous_daemon.py` および PowerShellコマンド実行知見

© 2026 N-Y-Public